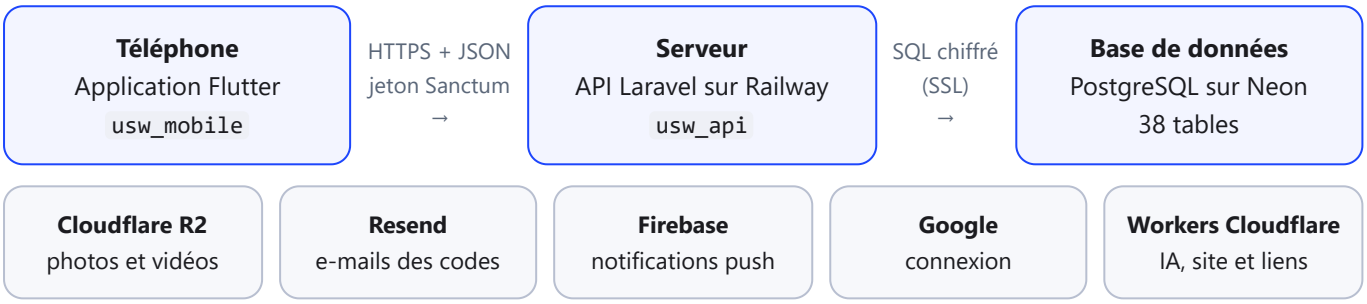
Discover : comment l'application, le serveur et la base de données sont reliés

Le trajet d'une donnée, du téléphone jusqu'à la base et retour, expliqué avec le vrai code du dépôt.
Dépôt cjdotdev/Discover · Application `usw_mobile` · Serveur `usw_api` · Version 2026

1. La vue d'ensemble

Discover est fait de trois blocs principaux et de quelques services extérieurs. Le téléphone ne parle jamais directement à la base de données : il passe toujours par le serveur.



Il y a deux exceptions à « tout passe par le serveur » :

- **Les photos et les vidéos** s'affichent depuis Cloudflare R2. Le serveur donne leur adresse, puis le téléphone les télécharge directement.
- **« Y aller »** ouvre l'application de cartes du téléphone avec les coordonnées du lieu. Le serveur n'intervient pas.

2. Qui fait quoi

Bloc	Technologie	Rôle	Où est le code
Application	Flutter 3.47, Dart	Écrans, appels à l'API, envoi des fichiers	`usw_mobile/lib`
Client API	Classe `Api`	Toutes les requêtes HTTP de l'app	`usw_mobile/lib/outils/api.dart`
Serveur	Laravel 13, PHP, FrankenPHP	Règles métier, sécurité, accès aux données	`usw_api/app`
Adresses de l'API	Routes Laravel	Chaque URL reliée à son contrôleur	`usw_api/routes/api.php`
Base de données	PostgreSQL (Neon)	Comptes, publications, lieux, réactions…	`usw_api/database/migrations`
Fichiers	Cloudflare R2 (compatible S3)	Photos, vidéos, images d'aperçu	`usw_api/app/Support/Medias.php`

Tâches de fond	File d'attente Laravel	Analyse des photos, vidéos, notifications	`usw_api/app/Jobs`
Hébergement	Railway (Docker)	Fait tourner le serveur 24 h sur 24	`usw_api/Dockerfile`

3. Première liaison : le téléphone et le serveur

Comment l'app trouve le serveur

L'adresse du serveur est écrite dans le code, avec une valeur par défaut. C'est pour ça qu'une app compilée par n'importe qui parle à notre serveur en ligne, sans réglage.

usw_mobile/lib/outils/api.dart

54	`static const base = String.fromEnvironment('API', defaultValue: 'https://discover-production-6cca.up.railway.app/api/v1');`	L'adresse de l'API. `String.fromEnvironment` permet de la changer au lancement (`--dart-define=API=…`). Sinon, c'est le serveur Railway.

212–219	`final requete = await _client.openUrl(methode, Uri.parse('$base/$chemin')).timeout(delai);`
	`requete.headers`
	`..set(HttpHeaders.acceptHeader, 'application/json')`
	`// La langue choisie : le serveur repond dans cette langue.`
	`..set(HttpHeaders.acceptLanguageHeader, Langue.instance.code);`
	`if (jeton != null) requete.headers.set(HttpHeaders.authorizationHeader, 'Bearer $jeton');`
	`if (appareil != null) requete.headers.set('X-Appareil', appareil!);`
	`entetes.forEach(requete.headers.set);`
		La requête part vers `base + chemin`, par exemple `…/api/v1/flux/decouverte`. Les en-têtes de **chaque** requête : on veut du JSON, la langue choisie, le **jeton** si le membre est connecté et un identifiant anonyme de l'appareil.

228–236	`final json = texte.isEmpty ? <String, dynamic>{} : (jsonDecode(texte) as Map).cast<String, dynamic>();`
	`if (reponse.statusCode >= 400) {`
	`// Le jeton n est plus valable (30 jours sans usage, deconnexion ailleurs).`
	`if (reponse.statusCode == 401 && jeton != null) await garderJeton(null);`
	`final erreurs = <String, String>{`
	`for (final e in ((json['erreurs'] as Map?) ?? const {}).entries)`
	`if (e.value is List && (e.value as List).isNotEmpty) e.key as String: (e.value as List).first as String,`
	`};`
	`throw ErreurApi(reponse.statusCode, json['message'] as String? ?? t('Une erreur s'est produite. Réessayez.'), erreurs);`
		La réponse est décodée de JSON en dictionnaire Dart. Une erreur (400 et plus) devient une `ErreurApi` avec son message ; sur 401, le jeton est oublié.

197–207	`    final refaisable = methode == 'GET' || methode == 'PUT' || methode == 'DELETE' || methode == 'HEAD';`
	`    if (!refaisable || brut != null) return _une(methode, chemin, corps: corps, brut: brut, type: type, entetes: entetes, delai: delai);`
	`    try {`
	`      return await _une(methode, chemin, corps: corps, type: type, entetes: entetes, delai: delai);`
	`    } on ApiInjoignable {`
	`      await Future<void>.delayed(const Duration(milliseconds: 800));`
	`    } on ErreurApi catch (e) {`
	`      if (e.code != 502 && e.code != 503 && e.code != 504) rethrow;`
	`      await Future<void>.delayed(const Duration(milliseconds: 800));`
	`    }`
	`    return _une(methode, chemin, corps: corps, type: type, entetes: entetes, delai: delai);`
		En cas de coupure ou de serveur qui redémarre, une lecture (GET, PUT, DELETE, HEAD) est refaite une fois, sans rien montrer au membre.

Le format des échanges

Toutes les réponses sont en JSON. Une erreur a toujours la même forme, ce qui permet à l'app de l'afficher sans cas particulier :

```
{ "message": "Le code est incorrect.", "erreurs": { "code": ["Code incorrect. Encore 2 essais."] } }
```

Code HTTP	Signification	Réaction de l'app
200 / 201	Réussi / créé	Affiche le résultat
401	Jeton absent ou expiré	Oublie le jeton : le membre se reconnecte
403	Interdit (pas son contenu, compte suspendu)	Affiche le message
409	Conflit (morceau d'envoi décalé)	Redemande au serveur où il en est
422	Données invalides	Affiche l'erreur sous le bon champ
429	Trop de requêtes	Réessaie plus tard
500 et plus	Panne du serveur	Garde le geste et le renvoie dès que le serveur répond

4. À l'intérieur du serveur : le trajet d'une requête

Une requête traverse toujours les mêmes étapes. Si l'une d'elles refuse, la requête s'arrête là et l'erreur repart au téléphone.

```
Requête HTTPS du téléphone
 → Railway (HTTPS) → FrankenPHP
 → bootstrap/app.php : les middlewares
     limite de débit · EnTetesSecurite · ChoisirLangue (accept-language) · NoterActivite
 → routes/api.php : l'adresse /api/v1/... trouve son contrôleur
 → auth:sanctum : qui est-ce ? (le jeton est cherché dans la base)
 → Form Request : les données sont-elles valides ?
 → Policy : a-t-il le droit de toucher à cet objet ?
 → Contrôleur → Service métier → Modèles Eloquent → PostgreSQL
 → Resource : l'objet devient du JSON
 → réponse au téléphone ; le travail long part dans la file d'attente
```

usw_api/bootstrap/app.php

16–17	`api: __DIR__.'/../routes/api.php',` `apiPrefix: 'api/v1',`	Toutes les routes de l'app commencent par `/api/v1`. Le « v1 » permettra une version 2 sans casser les anciennes apps.
23	`$middleware->trustProxies(at: '*');`	Railway place un relais HTTPS devant le serveur : on lui fait confiance pour que les adresses générées restent en https.
26	`$middleware->throttleApi('api');`	Une limite de requêtes par minute, contre les abus.
29	`$middleware->appendToGroup('api', ChoisirLangue::class);`	Le serveur répond dans la langue du téléphone (en-tête `accept-language`).

5. Deuxième liaison : le serveur et la base de données

La connexion à PostgreSQL

La base est sur **Neon**, un hébergeur PostgreSQL. Le serveur s'y connecte avec une adresse complète (utilisateur, mot de passe, hôte), rangée dans la variable d'environnement `DB_URL`. Elle n'est **jamais** dans le code ni dans le dépôt.

usw_api/config/database.php

21	`'default' => env('DB_CONNECTION', 'pgsql'),`	La base par défaut est PostgreSQL.

104–106	`'pgsql' => [`
	`'driver' => 'pgsql',`
	`'url' => env('DB_URL'),`
		`env()` lit les variables d'environnement de Railway : l'adresse de la base n'est pas dans le code.

117	`'sslmode' => env('DB_SSLMODE', 'require'),`	La connexion est chiffrée : Neon refuse les connexions en clair.

Neon passe par un **pooler** : il partage quelques connexions à la base entre toutes les requêtes du serveur.

Deux adresses vers la même base

Variable	Passe par	Sert à
`DB_URL`	Le pooler de Neon	Toutes les requêtes de l'app, nombreuses et courtes

`DB_URL_DIRECTE`	La connexion directe	Les migrations, que le pooler refuse

usw_api/docker/demarrer.sh

12	`DB_URL="${DB_URL_DIRECTE:-$DB_URL}" php artisan migrate --force`	À chaque déploiement, le serveur met la base à jour (`migrate`) par la connexion directe.

Du code PHP aux tables : Eloquent

Le serveur n'écrit presque jamais de SQL à la main. Chaque table a un **modèle** Eloquent, une classe PHP : `Utilisateur` pour `utilisateurs`, `Contenu` pour `contenus`… Le code écrit `Utilisateur::where('email', $email)->first()`, et Laravel le traduit en `SELECT * FROM utilisateurs WHERE email = ? LIMIT 1`.

Les **migrations** (`database/migrations`, 30 fichiers) décrivent chaque table en PHP. On peut ainsi recréer la base à l'identique n'importe où, et l'historique de sa forme est dans Git.

Les 38 tables, par rôle

Rôle	Tables
Comptes et connexion	utilisateurs, personal_access_tokens (jetons Sanctum), codes_email, sessions, appareils_push
Lieux	pays, villes, localites, lieux, lieux_enregistres
Publications	contenus, captures, modeles_telephone, fichiers_originaux, variantes, envois, etiquettes
Communauté	reactions, commentaires, jaimes_commentaires, collections, enregistrements, abonnements, notifications, vues, desinterets
Sons des reels	sons, sons_favoris, sons_signalements
Modération	signalements, blocages, decisions_moderation
Statistiques	activites_jour
Fonctionnement de Laravel	cache, cache_locks, jobs, job_batches, failed_jobs

La file d'attente et le cache sont aussi rangés dans PostgreSQL (tables `jobs` et `cache`) : pas besoin d'un serveur Redis en plus.

6. Troisième liaison : les fichiers

Une photo ne va pas dans la base : elle est trop lourde. La base garde seulement sa **description** (taille, adresse, appartenance), et le fichier va dans un stockage d'objets.

Disque	Où	Contenu
travail	Le disque du serveur	Un envoi en cours de réception, le temps de le vérifier

public	Cloudflare R2	Les versions affichées (WebP), servies à tous
originaux	Seau R2 privé	Les fichiers d'origine, si on choisit de les garder

Dans la réponse JSON d'une publication, le champ `images` donne les adresses R2 de chaque taille. Le téléphone les télécharge directement, sans charger le serveur.

usw_api/app/Http/Resources/ContenuResource.php

65	`'images' => $this->whenLoaded('variantes', fn () => $this->variantes->mapWithKeys(fn (Variante $v) => [`	Chaque variante (480, 1 080, 2 160 px…) devient une entrée avec son `url`, sa largeur et sa hauteur.
21	`'id' => $this->ulid,`	L'identifiant public est un ULID, et non le numéro de la ligne : on ne peut pas deviner les autres publications.

7. Trois parcours complets

Parcours 1 : se connecter avec un code reçu par e-mail

```
Téléphone                          Serveur                                  Base / services
1. e-mail saisi ──POST auth/code/envoyer──►  GestionnaireCodes          ──► codes_email (code haché, 10 min)
                                                                            ──► Resend envoie l'e-mail
2. code saisi  ──POST auth/code──────────►  vérifie le code (5 essais)   ──► utilisateurs (trouvé ou créé)
                                            Comptes::jeton()             ──► personal_access_tokens
3. ◄──── { jeton, utilisateur } ──────────
4. jeton gardé sur le téléphone (SharedPreferences), envoyé ensuite dans chaque requête
```

usw_mobile/lib/donnees/session.dart

131–132	`final r = await Api.instance.post('auth/code/envoyer', {'email': email, 'defi': ?_defi});` `_defi = r.json['defi'] as String? ?? _defi;`	Le téléphone demande un code. Le serveur renvoie un **défi**, un secret que seul ce téléphone garde : un autre ne peut pas essayer ce code.

175–178	`Future<void> ouvrir(Reponse r) async {`
	`await Api.instance.garderJeton(r.json['jeton'] as String?);`
	`_appliquer((r.json['utilisateur'] as Map).cast<String, dynamic>());`
	`connecte = true;`
		Après le bon code, le serveur renvoie le jeton : il est gardé, et le membre est connecté.

usw_api/app/Http/Controllers/Api/V1/AuthController.php

41–61	`    public function code(VerifierCode $requete): JsonResponse`
	`    {`
	`        $email = $requete->validated('email');`
	`        [$code, $echec, $reste] = $this->codes->verifier($email, $requete->validated('code'), GestionnaireCodes::BUT, $requete->validated('defi'));`
	`        if ($code === null) {`
	`            throw ValidationException::withMessages(['code' => match ($echec) {`
	`                'faux' => $reste === 1 ? 'Code incorrect. Encore 1 essai.' : "Code incorrect. Encore {$reste} essais.",`
	`                'epuise' => 'Trop d’essais : demandez un nouveau code.',`
	`                default => 'Ce code a expiré : demandez-en un nouveau.',`
	`            }]);`
	`        }`
	`        $code->update(['utilise_le' => now()]);`
	
	`        $u = Utilisateur::where('email', $email)->first();`
	`        $nouveau = $u === null;`
	`        $u ??= new Utilisateur(['email' => $email, 'fournisseur' => 'email']);`
	`        $u->email_verifie_le ??= now();`
	`        $u->nom_utilisateur ??= $this->comptes->nomLibre('membre'.random_int(10000, 99999));`
	`        $u->save();`
	
	`        return $this->connecte($u, $requete->validated('appareil'), $nouveau ? 201 : 200);`
		Le code juste ouvre le compte. S'il n'existe pas, il est créé (réponse 201). Un nom d'utilisateur libre est choisi.

usw_api/app/Services/Compte/Comptes.php

16–18	`    public function jeton(Utilisateur $u, ?string $appareil = null): string`
	`    {`
	`        return $u->createToken(Str::limit($appareil ?: 'application', 60, ''))->plainTextToken;`
		`createToken` (Laravel Sanctum) crée une ligne dans `personal_access_tokens` et renvoie le jeton en clair, une seule fois. La base n'en garde qu'une empreinte.

usw_api/app/Services/Compte/GestionnaireCodes.php

52	`        $code = (string) random_int(100000, 999999);`	Un code à six chiffres, tiré au hasard.
56	`            'code_hache' => Hash::make($code),`	Le code est **haché** avant d'aller dans la base : même avec la base, on ne peut pas le lire.
62	`            Mail::to($email)->send(new CodeParEmail($code, $but));`	L'e-mail part par Resend (variable `RESEND_API_KEY`, hors du dépôt).

Parcours 2 : afficher l'accueil

```
Téléphone ──GET flux/decouverte──► ExplorationController::decouverte
                                     page gardée 60 s en cache, avec ou sans compte
                                   Lieux + 4 meilleures photos ──► PostgreSQL
          ◄── { data: [lieux], meta: { next_cursor } } ──
Téléphone : construit les piles, puis télécharge les images depuis R2
```

usw_mobile/lib/donnees/catalogue.dart

275–280	`final r = await Api.instance.get('flux/decouverte');`
	`if (generation != _generation) return;`
	`final lieux = (r.json['data'] as List? ?? []).cast<Map>();`
	`_lieux = [for (final l in lieux) ? _lieuDe(l.cast<String, dynamic>())];`
	`_curseur = (r.json['meta'] as Map?)?['next_cursor'] as String?;`
	`_fin = _curseur == null;`
		Les lieux arrivent par pages de 20. Le **curseur** dit où reprendre pour la page suivante, quand le membre descend.

usw_api/app/Http/Controllers/Api/V1/ExplorationController.php

67–70	`$donnees = Cache::remember($cle, 60, function () use ($types, $requete) {`
	`$page = LieuResource::collection(`
	`$this->lieux($types)->where('slug', '!=', Lieu::SANS_LIEU)`
	`->orderByDesc('score')->orderByDesc('nombre_contenus')->orderByDesc('id')->cursorPaginate(20, cursorName: 'curseur')`
		La page est gardée 60 secondes en cache : 100 visiteurs en une minute ne font qu'une seule requête à la base. On peut regarder sans compte.

83	`$donnees['data'] = MaPart::dansLesLieux($donnees['data'] ?? [], $requete->user('sanctum'));`	Avec un jeton, on ajoute ce qui est propre au membre : sa réaction et ses enregistrements.

Parcours 3 : publier une photo

```
1. POST contenus           → crée le brouillon et deux envois (aperçu, original)
2. HEAD envois/{id}        → « où en es-tu ? » (Upload-Offset)
3. PATCH envois/{id} × n   → un morceau à la fois, repris après une coupure
4. POST envois/{id}/terminer → le fichier est complet
5. File d'attente : AnalyserCapture → signes d'IA, cohérence, variantes WebP → R2
6. statut « publié » → PrevenirAbonnes → EnvoyerPush (Firebase)
```

usw_mobile/lib/donnees/envois.dart

565–566	`final etat = await Api.instance.head('envois/$id');` `var decalage = int.tryParse(etat.entetes['upload-offset'] ?? '') ?? 0;`	Avant d'envoyer, le téléphone demande où en est le serveur : après une coupure, il reprend là, sans tout renvoyer.
575–576	`final r = await Api.instance.morceau('envois/$id', decalage, octets);` `decalage = int.tryParse(r.entetes['upload-offset'] ?? '') ?? decalage + octets.length;`	Chaque morceau part avec son décalage. Le serveur répond avec le nouveau décalage.
579–580	`if (err.code != 409) rethrow;` `decalage = int.tryParse((await Api.instance.head('envois/$id')).entetes['upload-offset'] ?? '') ?? 0;`	Erreur 409 : le serveur n'est pas au même point. Le téléphone lui redemande, puis continue.

589	`await Api.instance.post('envois/$id/terminer');`	Le fichier est complet : le serveur peut l'analyser.

usw_api/app/Services/Envois/Reception.php

41–42	`if ($decalage !== $envoi->recu) {` `throw new ErreurMetier(409, 'Le morceau ne suit pas le précédent.');`	Le serveur refuse un morceau qui ne suit pas le précédent : le fichier ne peut pas être mélangé.
48–49	`if ($n > self::MORCEAU_MAX || $envoi->recu + $n > $envoi->taille) {` `throw new ErreurMetier(413, 'Le fichier est trop gros.');`	Un morceau trop gros, ou un fichier plus grand qu'annoncé, est refusé.

usw_api/app/Http/Controllers/Api/V1/EnvoiController.php

63–64	`if (! $deja && $apercu && ($contenu->type === TypeContenu::Photo || $original)) {` `AnalyserCapture::dispatch($contenu);`	`dispatch` met l'analyse dans la **file d'attente** : le téléphone reçoit sa réponse tout de suite, et le travail lourd se fait en fond.

8. La sécurité des liaisons

- **Tout est chiffré :** HTTPS entre le téléphone et Railway, SSL entre le serveur et Neon.
- **Le jeton :** chaque requête d'un membre porte son jeton Sanctum, et le serveur sait ainsi qui parle. Se déconnecter supprime le jeton de ce téléphone.
- **Les droits :** les Policies (`ContenuPolicy`, `EnvoiPolicy`) vérifient qu'on ne modifie que ce qui nous appartient.
- **Les données reçues :** les Form Requests valident chaque champ avant tout accès à la base.
- **Les limites :** un nombre de requêtes par minute, et 5 essais pour un code.
- **Les secrets** (base, R2, Resend, Firebase, clé Laravel) sont des variables d'environnement de Railway. Aucun n'est dans le dépôt : `usw_api/.env.example` n'en donne que les noms.

9. Le déploiement

Étape	Ce qui se passe
Construction	Railway lit `railway.json` et construit l'image Docker de `usw_api/Dockerfile` : PHP, extension PostgreSQL, ffmpeg pour les vidéos.
Démarrage	`docker/demarrer.sh` migre la base, met la configuration en cache, puis lance le serveur web (FrankenPHP).
Tâches de fond	Dans le même conteneur, l'ouvrier de la file (`queue:work`) fait les analyses et les vidéos, et le planificateur (`schedule:work`) les tâches régulières.
Surveillance	Railway appelle `/up`. S'il ne répond pas, le service redémarre.

10. Faire tourner Discover chez soi

L'application seule, reliée à notre serveur

```
cd usw_mobile
flutter pub get
flutter run
```

L'app parle directement au serveur en ligne. Sa version (2026) est dans `pubspec.yaml`. Pour la présentation, on installe l'APK signé par l'équipe : la connexion Google, la carte et les notifications y marchent. Une app recompilée sans nos fichiers privés a ces limites :

- **Carte Google :** elle demande une clé dans `cles.json`, hors du dépôt.
- **Notifications push :** elles demandent `google-services.json`, hors du dépôt.
- **Connexion avec Google :** elle n'accepte que les apps signées avec notre clé. La connexion par e-mail marche partout.
- **App déjà installée :** l'app officielle et une app compilée chez soi portent le même nom de paquet (`bj.usw`), mais pas la même signature. Il faut désinstaller l'une pour installer l'autre.

Le serveur en local

```
cd usw_api
composer install
cp .env.example .env          # puis remplir les variables
php artisan key:generate
php artisan migrate
php artisan serve
```

Il faut alors une base PostgreSQL à soi, et lancer l'app avec `--dart-define=API=http://10.0.2.2:8000/api/v1` sur un émulateur Android.

11. Les questions probables

Pourquoi le téléphone ne parle-t-il pas directement à la base ?
Il faudrait mettre le mot de passe de la base dans l'app, et n'importe qui pourrait le lire. Le serveur garde les secrets, vérifie les droits et contrôle les données.

Comment le serveur sait-il qui fait la requête ?
Avec le jeton Sanctum envoyé dans l'en-tête `Authorization`. Le serveur retrouve sa ligne dans `personal_access_tokens`.

Pourquoi un pooler devant la base ?
Une base n'accepte qu'un nombre limité de connexions. Le pooler en partage quelques-unes entre toutes les requêtes.

Pourquoi les photos ne sont-elles pas dans la base ?
Elles sont lourdes. Un stockage d'objets comme R2 les sert vite et sans frais de sortie, et la base garde seulement leur description.

Que se passe-t-il si le réseau coupe pendant un envoi ?
Le téléphone demande au serveur où il en est (`HEAD`), puis reprend au bon morceau.

Pourquoi une file d'attente ?

Analyser une photo ou préparer une vidéo prend plusieurs secondes. Le téléphone reçoit sa réponse tout de suite, et le travailleur s'en occupe en fond.

Comment la base évolue-t-elle sans perdre de données ?

Par les migrations : chaque changement est un fichier, appliqué automatiquement au déploiement.

Où sont les mots de passe du serveur ?

Dans les variables d'environnement de Railway, jamais dans le code ni dans Git.